LAB 07 REPORT

Docker Setup and PostgreSQL Containers

Student Name	Malayka Ibrar
Course	DevOps
Lab Number	07
Operating System	Ubuntu Linux
Repository	DevOps (GitHub)

1. Introduction

In this lab, I worked with Docker on an Ubuntu server. I practiced checking the Docker installation, downloading Docker images, running containers, and managing PostgreSQL containers. I also used Docker commands to view images and container status and to remove a container and its image.

2. Objectives

- To check Docker and Docker Compose installation.
- To download and run PostgreSQL Docker images.
- To understand why PostgreSQL needs a password during first-time setup.
- To run and compare different PostgreSQL versions.
- To practice common Docker image and container commands.
- To restart, stop, and remove a container and remove its image.

3. Task 1: Docker Installation and Verification

Step 1: Open the Ubuntu terminal

I connected to the Ubuntu server through SSH and opened the terminal.

Step 2: Check the Docker version

I used the following command to check whether Docker was installed and to view its version:

```
docker --version
```

Step 3: Check Docker Compose

I checked the installed Docker Compose version using:

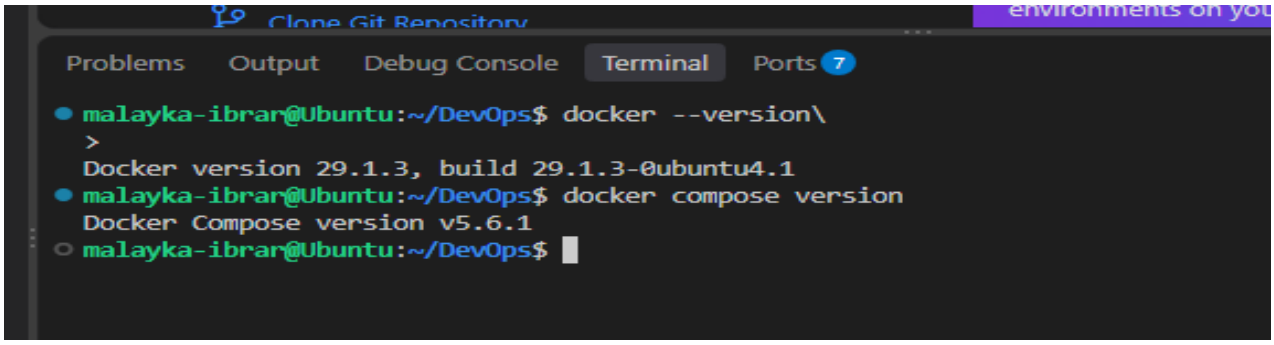
```
docker compose version
```

Step 4: View running containers

I used this command to check the containers that were currently running:

```
docker ps
```

Result: Docker and Docker Compose were available on the Ubuntu server. Docker was already installed, so a new installation was not required.



4. Task 2: PostgreSQL Docker Containers

Step 1: Download PostgreSQL 9.6

I downloaded the PostgreSQL 9.6 image from Docker Hub:

```
docker pull postgres:9.6
```

Step 2: Test PostgreSQL without setting a password

I tried to run the image without providing a password:

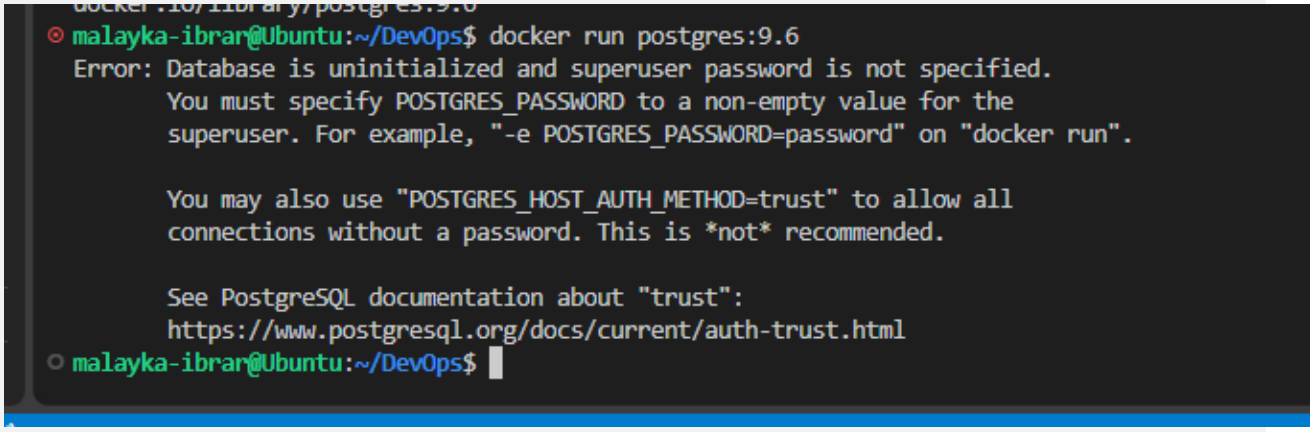
```
docker run postgres:9.6
```

The command displayed an error because the PostgreSQL image requires a password when initializing a new database.

Step 3: Run PostgreSQL 9.6 with a password

I ran the container in detached mode and set the required password:

```
docker run -d -e POSTGRES_PASSWORD=password postgres:9.6
```



Step 4: Check the running container

I used the following command to confirm that the PostgreSQL 9.6 container was running:

```
docker ps
```

Step 5: Download PostgreSQL 10.10

I downloaded another PostgreSQL version:

```
docker pull postgres:10.10
```

Step 6: Run PostgreSQL 10.10

I created and started a PostgreSQL 10.10 container with a password:

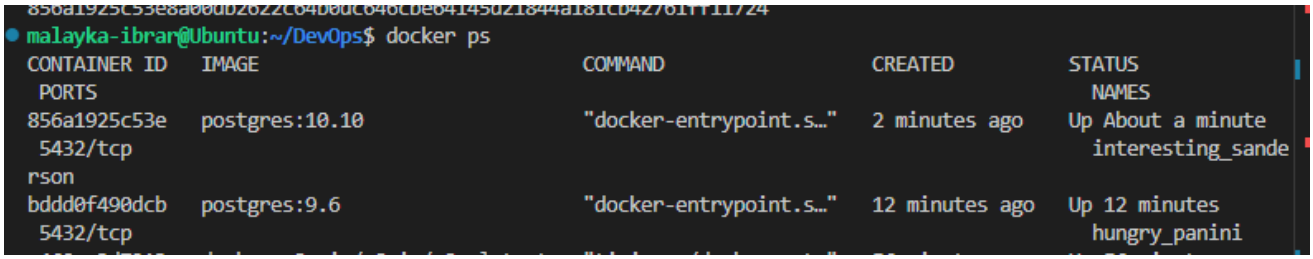
```
docker run -d -e POSTGRES_PASSWORD=password postgres:10.10
```

Step 7: Check both versions

I checked the running containers again to see both PostgreSQL versions:

```
docker ps
```

Result: PostgreSQL 9.6 and PostgreSQL 10.10 were downloaded and started as Docker containers.



5. Task 3: Docker Information and Ubuntu Container

Step 1: Display Docker information

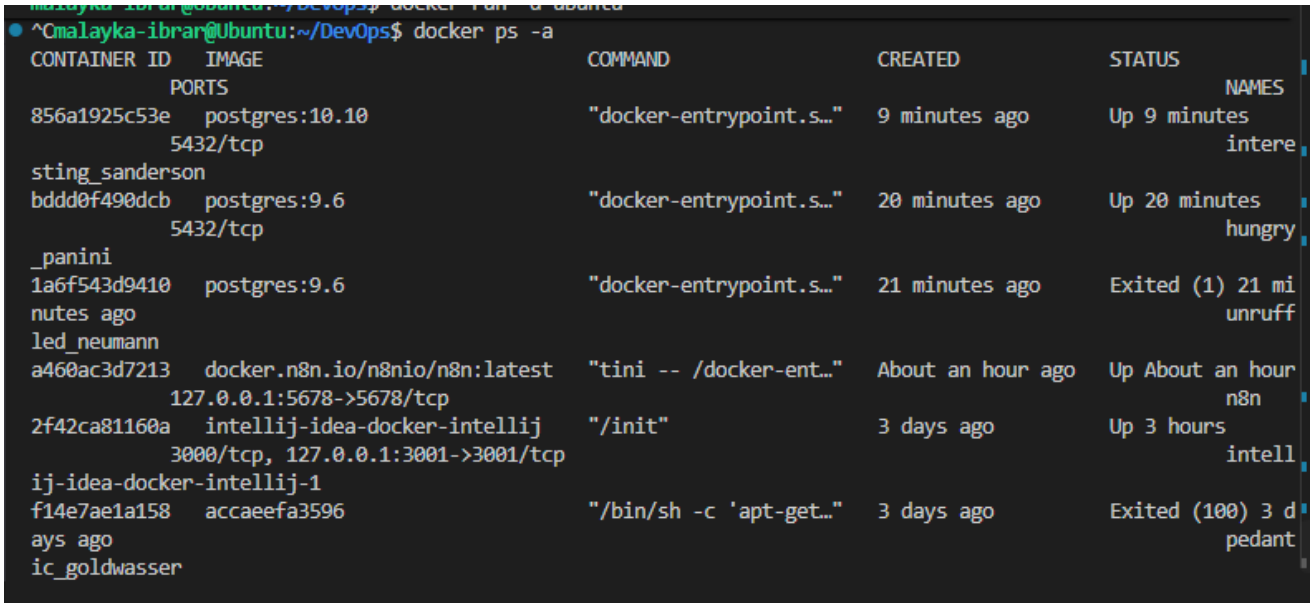
I used this command to view details about the Docker installation, server, images, and containers:

```
docker info
```

Step 2: Search for Ubuntu images

I searched Docker Hub for Ubuntu images using:

```
docker search ubuntu
```



Step 3: List downloaded images

I displayed the Docker images available locally:

```
docker images
```

Step 4: Run an Ubuntu container

I created an Ubuntu container in detached mode:

```
docker run -d ubuntu
```

The Ubuntu container can exit immediately because no long-running command was supplied to keep it active.

Step 5: View running containers

I used this command to display only running containers:

```
docker ps
```

Step 6: View all containers

I used this command to display running, stopped, and exited containers:

```
docker ps -a
```

Result: I practiced viewing Docker information, searching for an image, listing local images, and checking the status of containers.

6. Task 4: PostgreSQL 9.6 Container Cleanup

In this task, I practiced the basic cleanup operations for the PostgreSQL 9.6 container. I first identified the container name using docker ps -a. The container name can be different each time Docker creates a container, so the actual name shown on the terminal must be used in the commands.

Step 1: Identify the PostgreSQL 9.6 container

I checked all containers and noted the name of the container created from the postgres:9.6 image:

```
docker ps -a
```

Step 2: Restart the container

I restarted the PostgreSQL 9.6 container. I replaced the example name below with the actual container name:

```
docker restart <container-name>
```

Step 3: Stop the container

I stopped the same container:

```
docker stop <container-name>
```

Step 4: Remove the container

After stopping it, I removed the container:

```
docker rm <container-name>
```

Step 5: Remove the PostgreSQL 9.6 image

I removed the PostgreSQL 9.6 image from local Docker storage:

```
docker rmi postgres:9.6
```

Step 6: Verify the remaining containers and images

I checked the running containers and local images to verify the cleanup:

```
docker ps

docker images
```

Expected result: The PostgreSQL 9.6 container and image should be removed. PostgreSQL 10.10 should remain available and running. Other existing services should not be stopped or removed.

```
malayka-ibrar@Ubuntu:~/DevOps$ docker restart affectionate_edison
affectionate_edison
malayka-ibrar@Ubuntu:~/DevOps$ docker stop affectionate_edison
affectionate_edison
malayka-ibrar@Ubuntu:~/DevOps$ docker rm affectionate_edison
affectionate_edison
malayka-ibrar@Ubuntu:~/DevOps$ docker rmi postgres:9.6
Untagged: postgres:9.6
Deleted: sha256:caddd35b05cdd56c614ab1f674e63be778e0abdf54e71a7507ff3e28d4902698
malayka-ibrar@Ubuntu:~/DevOps$
```

```
malayka-ibrar@Ubuntu:~/DevOps$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          POR
TS                                                                     NAMES
bddd0f490dcb   postgres:9.6   "docker-entrypoint.s…"   13 seconds ago   Up 12 seconds   543
2/tcp                                                                  hungry_panini
```

```
malayka-ibrar@Ubuntu:~/DevOps$ docker info
   Profile: builtin
  cgroupns
 Kernel Version: 7.0.0-38-generic
 Operating System: Ubuntu 26.04.1 LTS
 OSType: linux
 Architecture: x86_64
 CPUs: 2
 Total Memory: 3.319GiB
 Name: Ubuntu
 ID: 60c42bb7-692e-4d17-ae73-93363b607118
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 Experimental: false
 Insecure Registries:
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false
 Firewall Backend: iptables

malayka-ibrar@Ubuntu:~/DevOps$
```

7. Conclusion

In this lab, I learned how to check Docker installation and use Docker commands on an Ubuntu server. I downloaded and ran two PostgreSQL versions, tested the password requirement, and practiced creating and inspecting an Ubuntu container. I also practiced restarting, stopping, and removing a PostgreSQL container and removing its image. This lab helped me understand the basic use of Docker images and containers.

8. Screenshots

The following screenshots are included as evidence of the commands and tasks performed in this lab. I will insert the screenshots in the spaces below.

Screenshot 1: Docker Installation

[Insert docker_installation.png here]

Screenshot space

Screenshot 2: PostgreSQL Missing Password Error

[Insert postgres_missing_password.png here]

Screenshot space

Screenshot 3: PostgreSQL 9.6 Running

[Insert postgres_96_running.png here]

Screenshot space

Screenshot 4: Both PostgreSQL Versions Running

[Insert postgres_both_versions.png here]

Screenshot space

Screenshot 5: Docker Information

[Insert docker_info.png here]

Screenshot space

Screenshot 6: Ubuntu Image Search

[Insert docker_search_ubuntu.png here]

Screenshot space

Screenshot 7: Ubuntu Container Status

[Insert ubuntu_container_status.png here]

Screenshot space

Screenshot 8: PostgreSQL 9.6 Cleanup

[Insert postgres_96_cleanup.png here]

Screenshot space